

Sisteme de operare

Laborator 12

Semestrul I 2025-2026

Laborator 11

- comunicare inter-procese (IPC)
 - programare client-server TCP/IP
 - clienti TCP si UDP
 - servere iterative TCP si UDP

Paradigma client-server

- paradigma generala de calcul
 - bazata pe mecanisme IPC
 - des intalnita in cadrul sistemelor NORMA bazate pe *message-passing*
 - in acelasi timp utila si in sisteme UMA/NUMA
 - ex: cozi de mesaje Sys V sau chiar sisteme de operare de tip microkernel (Mach)
 - defineste un serviciu executat de programe specializate numite *servere*
 - defineste un protocol de comunicatie prin care programe de aplicatie speciale, numite *client*, pot accesa serviciile serverului
 - protocolul de comunicatie este o forma de sincronizare intre procesele server si client
 - asigura coordonarea comunicatiei client-server

Context

- perspectiva de sistem de operare asupra unor servicii de comunicare inter-proces (IPC)
 - *pipes/FIFOs/cozi de mesaje SysV*
 - *sockets*
 - diferenta de paradigma: *shared memory* vs *message passing*
 - *pipe* – canal de comunicatie in memoria kernel (partajata) a aceleiasi masini
=> consecinte: kernelul asigura sincronizarea proceselor care scriu/citesc din *pipe*, la fel ca si politica de acces la date (FIFO)
 - *socket* – canal de comunicatie intre doua masini diferite legate intre ele printr-o retea de comunicatie
 - conecteaza doua endpoint-uri, i.e. perechi (adresa IP, port), o pereche pentru masina client, o pereche pentru masina server
- => consecinte: complexitatea operarii retelei de comunicatie (care poate pp implicarea mai multor masini intermediare pt. efectuarea schimbului de mesaje) impune folosirea unor *protocoale de comunicatie*

Ex protocoale de comunicatie

- TCP/IP

- asambleaza/dezasambleaza mesajele in pachete pt a putea fi transmise peste canale de comunicatie eterogene (i.e., care folosesc tehnologii de comunicare diferite) (TCP)
- asigura livrarea pachetelor in ordine la destinatie (TCP)
- rezolva automat probleme de rutare a pachetelor pe cai de comunicatie alternative intre nodurile sursa si destinatie atunci cand anumite masini intermediare nu sunt disponibile (IP)
- asigura retransmisia automata a pachetelor pierdute/defecte (TCP)
- controleaza congestia retelei (TCP)
- samd

Protocoale de comunicatie

- modelul OSI (Open Systems Interconnection)
 - 7 nivele: *fizic, data link, retea, transport, sesiune, prezentare, aplicatie*
- model alternativ, protocoalele internet, a.k.a. protocoalele DoD (Department of Defense):
 - combina ultimele trei nivele OSI intr-un singur nivel: *aplicatie*
 - combina nivelul fizic si data link intr-un singur nivel: *link*
- conceptual, functioneaza pe principiul *stivei de protocoale*: protocolul de nivel cel mai inalt (*aplicatie*) apeleaza la serviciile protocolului imediat anterior (*prezentare*), samd
 - la nivelul cel mai de jos, datele trimise de protocoalele *data link* sunt trimise peste reseaua fizica
 - la receptie se parcurge calea inversa
- exemple protocoale:
 - data link (retele LAN): Ethernet (CSMA/CD), Token Ring, ATM, Wireless
 - retea: IP
 - transport de date: TCP, UDP, SCTP
 - sesiune: RPC
 - prezentarea datelor: XDR, ASN.1, criptare, compresie
 - aplicatie: DNS, SMTP, HTTP, FTP, SSH, etc

Protocoale de comunicatie (cont.)

- curs de retele de calculatoare: studiul intregii stive de protocoale OSI/internet
- laborator SO:
 - studiem protocoalele de comunicatie din perspectiva IPC
 - ne marginim la protocoale de comunicatie de nivel aplicatie
 - folosim modelul internet
- consecinta: vom folosi abstractia de *socket* pentru a crea cai de comunicatie bazate pe protocoalele de nivel transport ale internetului: TCP/UDP/SCTP
- **socket**
 - endpoint local al caii de comunicatie
 - tuplu (adr IP, port)
 - creat cu apelul sistem *socket*

int socket(int domain, int type, int protocol);

TCP sockets

- creeaza un canal de comunicatie (o conexiune) **sigur (reliable)** intre doua endpoint-uri (adr IP, port)
- canalul de comunicatie garanteaza livrarea corecta si in ordine a mesajelor intre sursa si destinatie
- mesajele pierdute se retransmit automat
- canalul de comunicatie are o semantica asociata de tip stream de octeti (analog *pipe*)
 - ca atare, din socket-ul TCP se citesc octeti, nu mesaje intregi !
- se creeaza folosind constanta predefinita `SOCK_STREAM` in cel de-al doilea parametru al apelului sistem *socket*
- un socket TCP trebuie intotdeauna conectat folosind apelul sistem *connect*

UDP sockets

- creeaza un canal de comunicatie *fara garantii* (**nesigur**, **unreliable**) intre doua endpoint-uri
- mesajele (s.n. *datagrame*, ca la protocolul IP) se pot pierde, nu se retransmit automat, nu se livreaza in ordine
- mesajele se citesc in intregime, i.e. o datagrama la un moment dat (spre deosebire de TCP)
- se creeaza folosind constanta predefinita `SOCK_DGRAM` in apelul sistem *socket*
- in general, un socket UDP nu trebuie conectat
 - exista primitive specifice bibliotecii de sockets pt transmisia/receptia mesajelor (datagramelor): *sendmsg/sendto* respectiv *recvmsg/recvfrom*
 - aceste apeluri sistem includ endpoint-ul sursa/destinatie printre parametri
- daca se foloseste *connect* pt un socket UDP:
 - se creeaza o asociere intre endpoint-urile client si server care permite folosirea apelurilor sistem de file I/O *read/write* (si variantele lor)
 - momentan, vom folosi UDP sockets conectati pentru a putea folosi *read/write*

Client-server cu sockets

- client-server TCP/IP
 - serverul creeaza un socket *pasiv* folosit pentru a astepta cereri de conexiune de la client
 - clientul creaza un socket *activ* prin care se conecteaza la server cu ajutorul apelului sistem *connect*

```
int connect(int sd, const struct sockaddr *addr, socklen_t addrlen);
```

- serverul accepta mai intai cererea de conexiune si apoi trateaza cererea
- client-server UDP/IP
 - clientul si serverul comunica direct folosind sockets cu ajutorul unor primitive specifice de tip *send/receive*
- *Obs:* mesajele transmise intre clienti si servere trebuie transformate in *network byte order* (v. curs RPC)

Server sockets

- apeluri sistem specifice
- **bind**
 - la creare, un socket nu are asociat un endpoint (nu are asociate adrese, nici locale nici distante)
 - specifica un endpoint pt un socket din programul apelant
 - asociaza un socket descriptor cu o adresa de endpoint (*sockaddr_in* pt TCP/IP, contine adresa IP si port)
 - serverele folosesc *bind* pt a specifica portul (*well-known port*) pe care asteapta cereri de la client

```
int bind(int sd, const struct sockaddr *addr, socklen_t addrlen);
```

TCP server sockets - listen

- la creare, un socket nu este nici activ (gata de folosinta pt un client), nici pasiv (gata de utilizare pentru un server)
- serverele orientate pe conexiune (TCP) trebuie sa puna socket-ul in mod pasiv
 - apel sistem **listen**
 - socket-ul devine disponibil pt a accepta cereri de conexiune

int listen(int sd, int backlog);

- model de calcul servere TCP: loop infinit in care
 - accepta urmatoarea cerere de conexiune
 - trateaza cererea
 - revine in bucla pentru a asteapta urmatoarea cerere
- oricat de scurta ar fi tratarea unei cereri, e posibil ca o noua cerere sa vina in timpul tratarii cererii curente
 - apare posibilitatea pierderii unei cereri
 - *listen* instruieste SO sa puna cererile adresate unui socket si netratate inca intr-o coada (asa numitul *backlog*)
 - dimensiunea backlog-ului este pasata ca parametru apelului *listen*

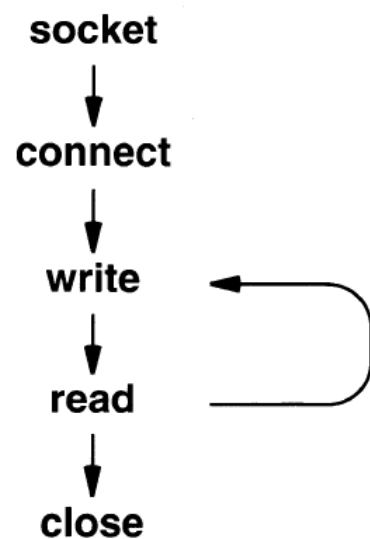
TCP server sockets - accept

- Q: cum se accepta noi cereri de conexiune?
- A: apel sistem **accept**
 - primește ca parametru socket-ul pt care trebuie acceptate conexiuni
 - creează un nou socket pt fiecare cerere nouă de conexiune
 - întoarce ca valoare descriptorul noului socket creat
 - socket-ul nou creat este folosit doar pentru noua conexiune creată
 - socket-ul original e folosit pt a accepta noi cereri de conexiune
 - după acceptarea unei cereri de conexiune, serverul transferă date folosind socket-ul nou
 - la finalul tratării cererilor clientului, serverul închide socket-ul nou creat cu *close*

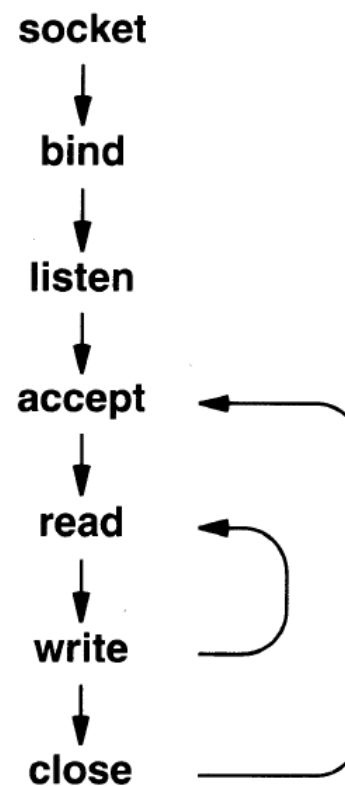
*int accept(int sd, struct sockaddr *addr, socklen_t *addrlen);*

Utilizarea TCP sockets in programe

CLIENT SIDE



SERVER SIDE



Exemple de protocoale simple

- *daytime, echo, time*
- comenzile de tip *telnet host port* emuleaza activitatea clientului
- pentru partea de server, la inceput folosim servicii predefinite
- serverele standard pornite cu *inetd/xinetd*
 - server cu structura particulara care multiplexeaza mai multe servicii diferite
 - serviciile multiplexate de server se configureaza cu ajutorul unor fisiere de configurare aflate in */etc/xinetd.d*, specifice fiecarui serviciu
 - in general, serverele au fisiere de configurare, clientii au doar parametri de apel in linia de comanda
 - diferenta e dictata de complexitatea diferita a celor doua tipuri de programe
- serviciile standard in Linux gestionate cu comanda *service* (necesita drepturi de root)

\$ service <serviciu> <start | stop | reload>
- modificarile fisierelor de configurare ale serverelor sunt luate in considerare doar dupa “reincarcarea” serviciului (*service reload*)
 - alternativa: *inotify*

Exemplu HTTP

- telnet <host> 80

- apoi comanda

GET / HTTP/1.1

(sau GET /index.html HTTP/1.1)

Host: <host>

<cr/lf>

<cr/lf>

- telnet fmi.unibuc.ro 80 ?

Sisteme de operare

Laborator 12

Cienti si servere iterative TCP/UDP

1. Creati un nou subdirector *lab12/* in structura de directoare a laboratorului creata anterior (*SO/laborator*) si subdirectoarele aferente *doc src* si *bin*. Nu uitati sa actualizati variabila de mediu *PATH* pentru a include directorul *SO/laborator/lab12/bin*.

2. Instalati pe calculator superserverul internet (**inetd** / **xinetd**). Activati serviciile TCP de **daytime** si **echo**, respectiv serviciul UDP **time** (*Indicatie*: setati campul *disable* pe *no* in fisierele de configurare ale serviciilor aflate in directorul */etc/xinetd.d* si apoi folositi comanda *service* ca superuser pentru a reincarca configuratia serviciului corespunzator). Cautati in fisierul */etc/services* porturile pe care functioneaza aceste servicii si incercati sa vedeti cum functioneaza serviciile folosind urmatoarea comanda

\$ telnet localhost <port>

Obs: comanda de mai sus nu functioneaza decat peste TCP, adica nu veti putea testa decat functionalitatile serviciilor TCP pe care le-ati activat in fisierele de configurare.

3. Folositi apelurile sistem *socket* si *connect* pentru a scrie programe client **daytime-tcp.c**, **echo-tcp.c** si respectiv **time-udp.c** care acceseaza serviciile corespunzatoare activate cf exercitiului anterior.

Clientul **echo-tcp.c** citeste in bucla date de la tastatura pe care le trimite catre server si afiseaza raspunsul serverului inainte de a incepe urmatoarea iteratie. In plus, clientul verifica daca raspunsul serverului este acelasi cu mesajul pe care l-a trimis, atat in privinta lungimii cat si a continutului.

Clientul **daytime-tcp.c** se conecteaza la server dupa care asteapta raspunsul serverului si il tipareste pe ecran.

Clientul **time-udp.c** se conecteaza la server dupa care ii trimite mesajul *"what time is it?\n"*. Apoi asteapta raspunsul serverului si il tipareste pe ecran.

Obs: Protocolul **daytime** foloseste siruri de caractere pentru codificarea timpului, in vreme ce protocolul **time** codifica timpul intr-un intreg de tip *long*.

Indicatii: timpul in Unix este reprezentat ca numar de secunde incepand de la 1 ian. 1970. Pentru a converti timpul obtinut cu apelul sistem *time* la timpul curent folositi urmatoarea secventa de cod:

```
#define UNIXEPOCH 2208988800
```

```
time_t now;
```

```
now = time(NULL);  
now -= UNIXEPOCH;
```

Folositi functia de biblioteca *ctime* pentru a afisa o valoare de tip *time_t*.

La apelul *socket* puteti folosi ca protocol valoarea 0 (al treilea argument de apel). Pentru a putea efectua apelul *connect*, completati campurile structurii urmatoare in mod corespunzator:

```

struct sockaddr_in
{
    short sin_family; // AF_INET
    unsigned short integer sin_port; // portul setat cu htons
    struct in_addr sin_addr;
    char sin_zero[8];
};

struct in_addr
{
    unsigned long integer s_addr; // folositi inet_addr / inet_aton
                                     // pt a seta adresa de internet
};

```

Programele se vor compila si executa dupa cum urmeaza:

```
$ gcc -o daytime-tcp daytime_tcp.c
```

```
$ daytime-tcp 127.0.0.1 13 # adresa de IP si portul de serviciu
```

Obs: pentru acest laborator vom folosi numai clienti UDP conectati (adica programe care apeleaza *connect* pentru sockets UDP pentru a putea folosi primitivele uzuale de citire/scriere pentru fisiere, *read/write*).

4. Scrieti un server TCP **echod-tcp.c** pentru serviciul de **echo** care sa raspunda la cererile clientului TCP de mai sus. Serverul **echod-tcp.c** trimite in ecou caracter cu caracter mesajul primit de la client. (*Indicatie:* in acest scop, puteti folosi functia *myprint* din programul **race.c** din laboratoarele trecute).

Ce consecinta are comportamentul serverului asupra felului in care trebuie scris clientul de **echo** de mai sus?

5. Scrieti un server UDP **timed-udp.c** pentru serviciul de **time** care sa raspunda la cererile clientului UDP de mai sus. Pentru receptia datagramelor UDP folositi apelul sistem *recvfrom*, iar pentru transmitia datagramelor UDP folositi apelul sistem *sendto* (*Obs:* serverul nu poate folosi apelul sistem *connect* la fel ca un client, asa ca nu poate nici folosi apelurile sistem *read/write* pentru a primi si respectiv a trimite mesaje).

Ce observati prin comparatie ca diferenta intre receptia datelor in serviciul TCP de **echo** fata de receptia datelor in serviciul UDP de **time**?

Indicatie: pt a afla timpul, serverul poate folosi apelul sistem *time* respectand conventiile Unix referitoare la timp explicate mai sus.

6. Scrieti un server TCP **daytimed-tcp.c** pentru serviciul de **daytime** care sa raspunda la cererile clientului TCP de mai sus.

7. Folositi un shell script pentru a verifica functionalitatea iterativa a serverului **echod-tcp** de mai sus. In acest scop trebuie sa modificati apelul clientului TCP **echo-tcp** de mai sus a.i. sa nu mai citeasca datele de la tastatura ci dintr-un fisier text aflat in directorul curent (de pilda codul sursa al clientului sau al serverului, **echo-tcp.c** sau **echod-tcp.c**).

Indicatii: Nu trebuie sa modificati codul clientului de mai sus, e suficient sa redirectati din shell file descriptorul corespunzator pentru a putea cititi datele din fisier.

Pentru a verifica iterativitatea serviciului puteti lansa in executie mai multi clienti **echo-tcp** in paralel, de pilda lansand comenzile din script in background ca mai jos:

```
#!/bin/sh
```

```
echo-tcp ... &
```

```
echo-tcp ... &
```

```
...
```

```
echo-tcp ... &
```